



Core Java

Classes and Object

Outline

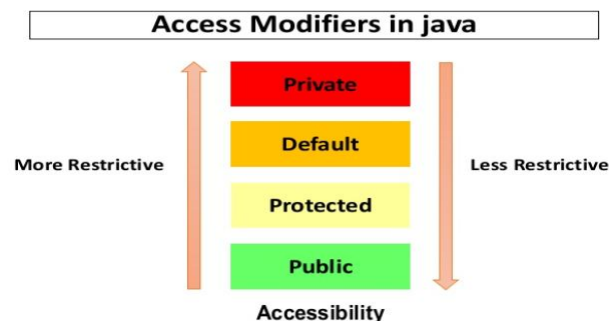
- Classes
- Access properties and methods
- this keyword
- Reference type.
- Super Classes & Subclasses
- types of inheritance supported by Java
- Overloading and Overriding of methods
- final keyword
- member and method visibility in inheritance.
- Object – Superclass,
- Wrapper Classes
- Object Wrappers and Autoboxing
- Method with Variable Number of Parameters.

Outline

- Abstract Classes
- Enumeration Classes
- Reflection.
- Interface Concept, Properties of Interfaces, Interfaces and Abstract Classes, Static and Private Method (in Interface), Default Methods, Interfaces and Callbacks.
- Functional Interface - Introduction, Sealed Classes and Interface, Pattern Matching - Switch Expression and instanceOf.
- Object Cloning, The Comparator Interface, Inner Classes, Anonymous inner class, Static Class.
- Records: Introduction, Constructors - Canonical, Compact, and Custom, Getter Methods

Access properties and methods

- The access specifiers (also known as access modifiers) used to restrict the scope or visibility or accessibility of a class, constructor, variable, method or data member of class and interface.
- There are four access specifiers, and their list is below.
 - default (or) no modifier
 - public
 - protected
 - private



Access properties and methods

Access control for members of class and interface in java

Accessibility Location Access Specifier	Same Class	Same Package		Other Package	
		Child class	Non-child class	Child class	Non-child class
Public	Yes	Yes	Yes	Yes	Yes
Protected	Yes	Yes	Yes	Yes	No
Default	Yes	Yes	Yes	No	No
Private	Yes	No	No	No	No

Where Access specifier can be applied

Access Specifier ► Item ▼	Default	Public	Protected	Private
Class	Yes	Yes	No	No
Inner Class	Yes	Yes	Yes	Yes
Interface	Yes	Yes	No	No
Interface Inside Class	Yes	Yes	Yes	Yes
enum	Yes	Yes	No	No
enum Inside Class	Yes	Yes	Yes	Yes
enum inside Interface	Yes	No	No	No
Constructor	Yes	Yes	Yes	Yes
methods & data inside class	Yes	Yes	Yes	Yes
methods & data inside Interface	Yes	No	No	No

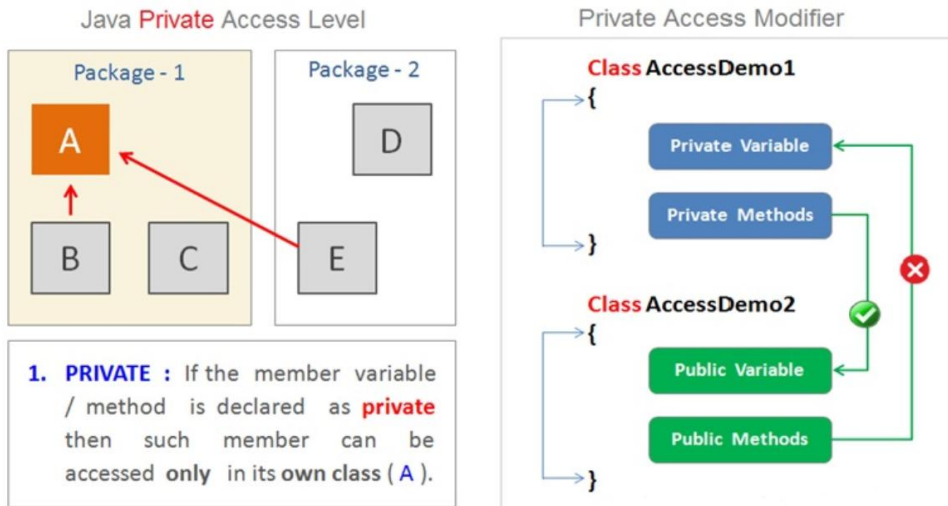
Access properties and methods

- The public members can be accessed everywhere.
- The private members can be accessed only inside the same class.
- The protected members are accessible to every child class (same package or other packages).
- The default members are accessible within the same package but not outside the package.

Private Access modifier

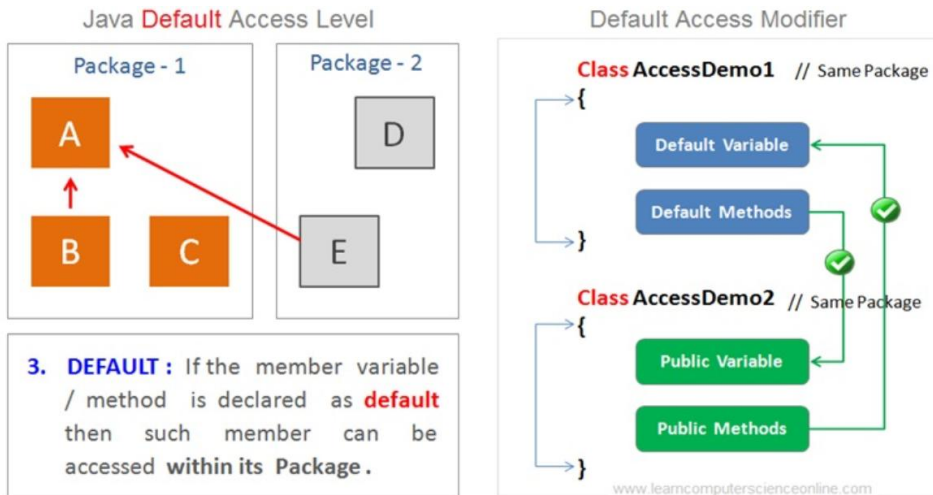
- Is the most restrictive modifier which allows access only within the class.
- Which means the members of subclass, other classes in the same package and also the members of from other packages do not get access to the private member.
- If the class member method or a variable is declared as **private** then such member will be accessible **only** with its **own class**. A class (outer class) cannot be declared as private.
- However, an inner class (a class within class) can be declared as private. The class members such as methods and variables can be declared as private.

Access properties and methods



Default Access modifier

- It will be applicable when we do not specify any access modifier while declaring a class, method or variable then the access level is said to be the default.
- It is the second most restrictive modifier which allows access only within the package.
- The members of the same class as well as other classes in the same package can only have access to the default members.
- But the members of the other packages and also the subclasses does not get access to the default member.



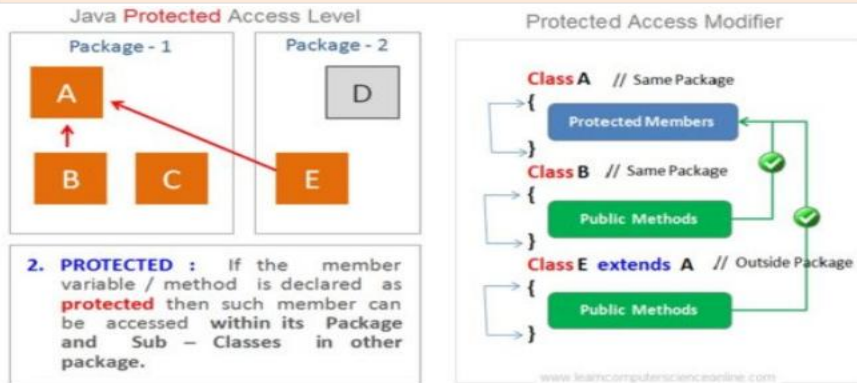
As explained in this example the class A , class B and class C belongs to the **same package** (package 1) .
And hence , the members of class B and class C can get access to the default members of class A .

Whereas the class D and class E belong to the **different package** (package 2) and hence cannot get access to the default members of class A .

Protected Access modifier

- It will be applicable when we specify protected access modifier while declaring a method or variable.
- It is the third most restrictive modifier which allows access within the package and also to the members of the subclasses in other packages.

Protected Access modifier



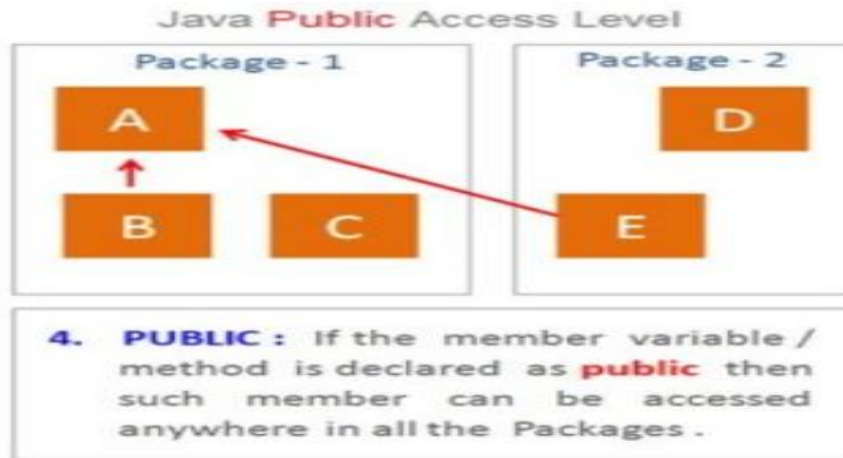
As explained in this example the class A, class B and class C belongs to the **same package** (package 1) .
And hence , the members of class B and class C can get access to the protected members of class A .

The class E **extends** class A which is a **sub class** of class A and therefore will get access to the protected members of class A . Whereas the class D belong to the **different package** (package 2) and hence cannot get access to the protected members of class A .

Public Access modifier

- It will be applicable when we specify public access modifier by using public keyword while declaring a class, method or variable.
- It is the least restrictive modifier which allows access within the package and also to the members of the subclasses in other packages.
- It is the most liberal access and provides maximum visibility under which a public member can be accessed anywhere in the java project.

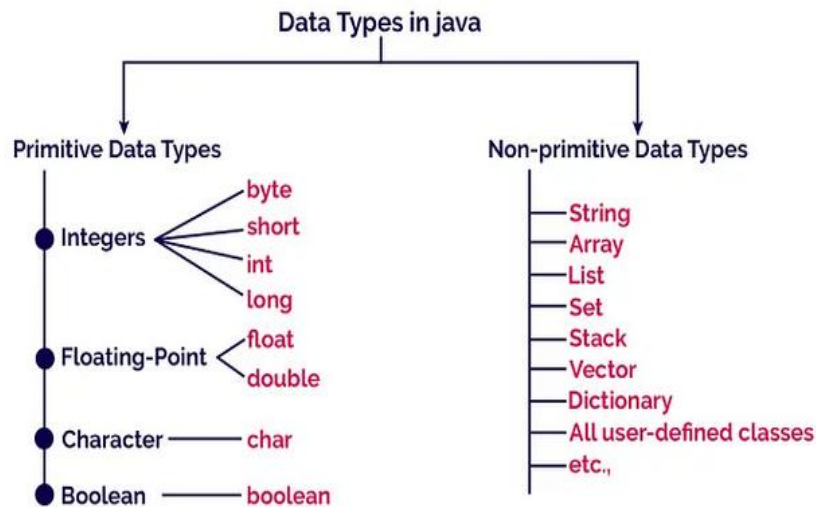
Public Access modifier



Java Keywords

Category	Java Keywords
Access Modifiers	private, protected, public
Class, Method, Variable Modifiers	abstract, class, extends, final, implements, interface, native, new, static, strictfp, synchronized, transient, volatile
Flow Control	break, case, continue, default, do, else, for, if, instanceof, return, switch, while
Package Control	import, package
Primitive Types	boolean, byte, char, double, float, int, long, short
Error Handling	assert, catch, finally, throw, throws, try
Enumeration	enum
Others	super, this, void
Unused	const, goto

Data types



Data types

- **Primitive Types** : Primitive data types are pre-defined in Java such that they come from language itself directly.
- We use primitive types as building blocks to create reference type.
- It's important to note that String data type is predefined but it's a reference type because it uses char data type as building block and defines useful methods to manipulate the string.
- **Reference Types (Non-primitive Types)** : Reference types are not pre-defined, instead they are created by the programmer using the primitive types.
- The "reference" in the name comes from the fact that reference types hold references to actual objects.
- Class, Interface, Array, Enumeration and Annotation are all reference types in Java.

Key Differences Between Primitive Types and Reference Types

- Primitive types are pre-defined. Reference types are not pre-defined except String.
- All primitive types begin with lowercase letter. Reference types are preferred to begin with uppercase letter.
- Primitive types cannot be null. Reference types can be null.
- Primitive types are allocated in stack memory. Reference types are allocated in heap memory.
- Primitive types are deallocated from the memory when they are out of the scope. Reference types are deallocated from the memory by garbage collector when there are no references to them.

Conversion Between Primitive and Reference Type

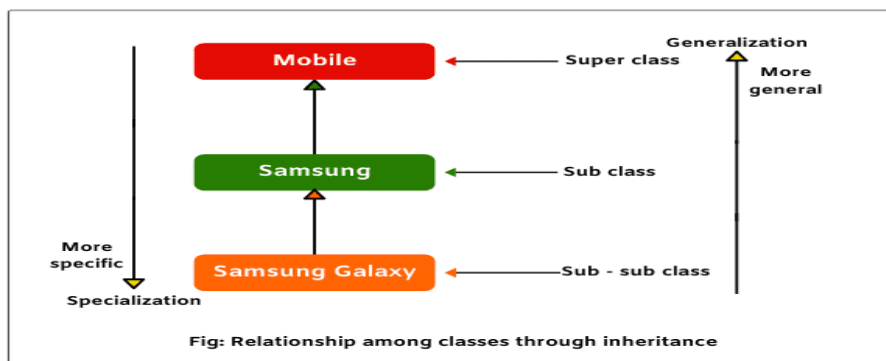
- **Autoboxing** : The conversion of primitive type to reference type(box) is called autoboxing. It's generally done by using conversion constructors or class/object methods.
- **Unboxing**: The conversion of reference type to primitive type is called unboxing.

Comparing Primitive Type Variables and Reference Type Variables

- We can compare primitive type variables using equal-to operator(==).
- We can compare reference type variables in the same way with using equal-to operator(==) but the meaning is different this time. (==) compares the references of the variables so it is only true when both sides are the exact same object. If we want to compare the value/content of the reference type variables then we need to use “compareTo” method.

Super Class and subclasses

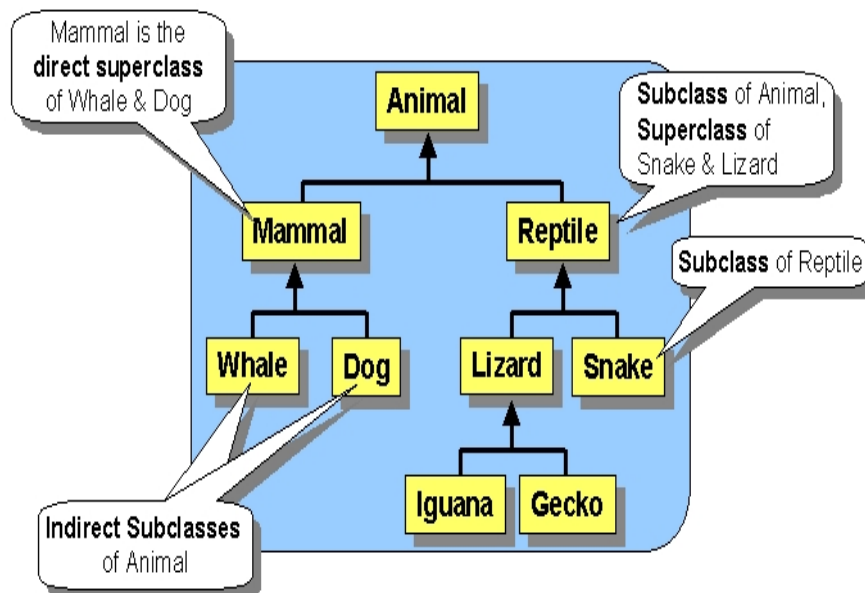
- **Generalization** : The process of converting subclass type into superclass type is called **generalization in Java**. This is because we are making the subclass to become more general so that its scope can be more widening.
- This conversion is also called widening or upcasting in referenced data types.



Super Class and subclasses

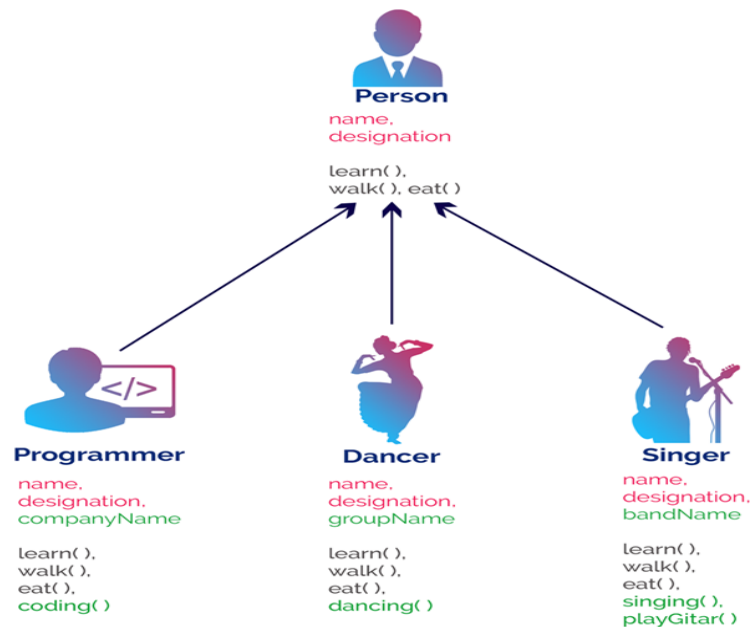
- **Specialization** : The conversion of a superclass type into subclass type is called specialization in java.
- Specialization means going down from a more general form to a more specific form.
- Thus, its scope will be narrowed. Hence, this conversion is also called narrowing or down-casting in referenced data types.
- Super class : A class that is used to create a new class is called **superclass in Java**. In other words, the class from where a subclass inherits the features is called superclass. It is also called a base class or parent class in Java.
- Subclass : A class that inherits all the members (fields, method, and nested classes) from the other class is called **subclass in Java**. In simple words, a newly created class is called subclass. It is also called a derived class, child class, or extended class in Java.

Super Class and subclasses



Inheritance

- Using the inheritance concept, we can use the existing features of one class in another class. The inheritance provides a greater advantage called code re-usability.
- The mechanism of a class to derive properties and characteristics from another class is called Inheritance.
- Inheritance is the process, by which class can acquire (reuse) the properties and methods of another class.
- **Base Class:** The class whose properties are inherited by sub class is called Base Class/Super class/Parent class.
- **Derived Class:** The class that inherits properties from another class is called Sub class/Derived Class/Child class.
- Inheritance is implemented using super class and sub class
- relationship in object-oriented languages.
- Inheritance is also known as “**IS-A relationship**” between parent and child classes.



How to implement Inheritance

- To inherit a class, you simply incorporate the definition of one class into another by using the **extends** keyword.
- Syntax:**

```
class subclass-name extends superclass-name
{
    // body of class...
}
```

```
class A
{
    //SuperClass or ParentClass or BaseClass
}
```

the keyword "**extends**" is used to create a subclass of **A**

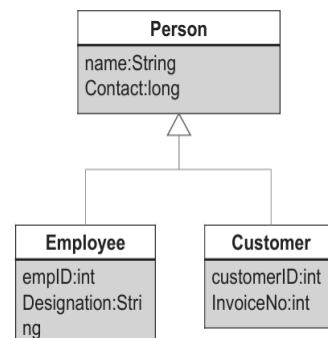
```
class B extends A
{
    //SubClass or ChildClass or DerivedClass
}
```

Implement Inheritance example

```
1. class Person
2. {
3.     String name;
4.     long contact;
5. }

6. class Employee extends Person
7. {
8.     int empID;
9.     String designation;
10.}

11. class Customer extends Person
12. {
13.     int customerID;
14.     int invoiceNo;
15.}
```

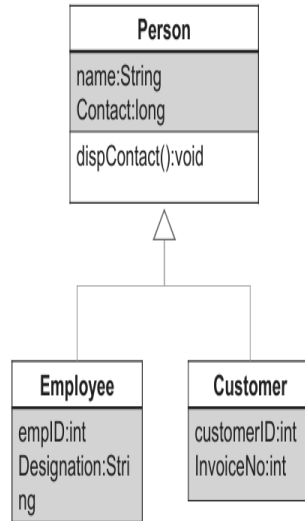


Implement Inheritance example

```

1. class Person
2. {
3.     String name;
4.     long contact;
5.     public void dispContact()
6.     { System.out.println("num="+contact);
7.     }
8. }
9. class Employee extends Person
10. {
11.     int empID;
12.     String designation;
13. }
14. class Customer extends Person
15. {
16.     int customerID;
17.     int invoiceNo;
18. }

```



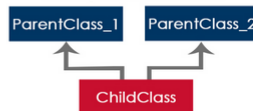
Types of Inheritance

- Simple Inheritance (or) Single Inheritance
- Multiple Inheritance
- Multi-Level Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

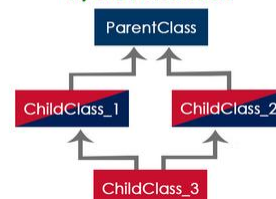
Simple Inheritance



Multiple Inheritance



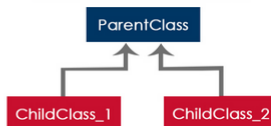
Hybrid Inheritance



Multi Level Inheritance



Hierarchical Inheritance



The java programming language does not support multiple inheritance type. However, it provides an alternate with the concept of interfaces.

Single Inheritance

- One child class derives from one parent class

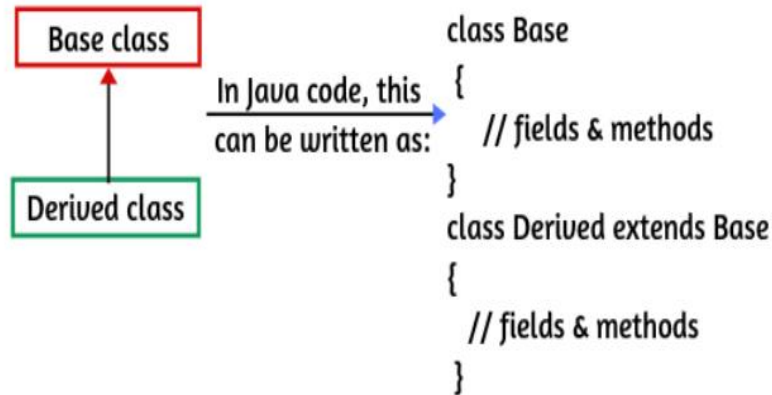


Fig: Single level Inheritance

Single Inheritance example

```

class ParentClass{
    int a;
    void setData(int a) {
        this.a = a;
    }
}
class ChildClass extends
ParentClass{
    void showData() {
        System.out.println("Value of a
is " + a);
    }
}

```

```

public class SingleInheritance {
    public static void
main(String[] args) {
        ChildClass obj =
new ChildClass();
        obj.setData(100);
        obj.showData();
    }
}

```

```

<terminated> SingleInheritance.java applicati
Value of a is 100

```


Multi level Inheritance

- The child class derives from a class which already derived from another class

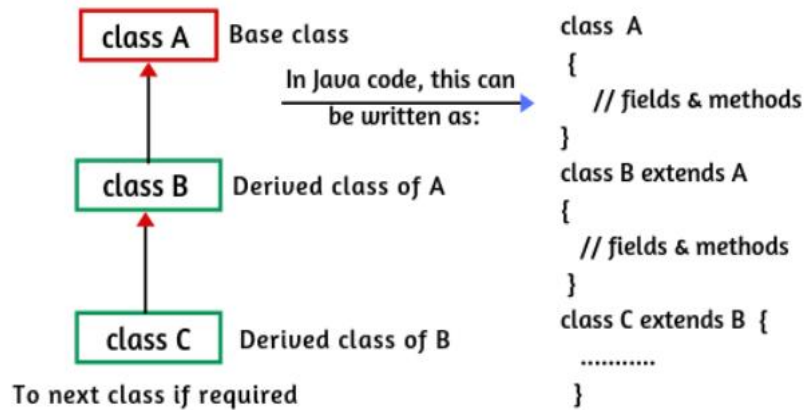


Fig: Multilevel Inheritance in Java

Multi level Inheritance example

```

class ParentClass{
    int a;
    void setData(int a) {
        this.a = a;
    }
}
class ChildClass extends
    ParentClass{
    void showData() {
        System.out.println("Value
        of a is " + a);
    }
}
  
```

```

class ChildChildClass extends
    ChildClass{
    void display() {
        System.out.println("Inside
        ChildChildClass!"); }
}
public class MultipleInheritance {
    public static void
    main(String[] args) {
        ChildChildClass obj = new
        ChildChildClass();
        obj.setData(100);
        obj.showData();
        obj.display();}
}
  
```

Value of a is 100
Inside ChildChildClass!

Hierarchical Inheritance

- Two or more child classes derive from one parent class

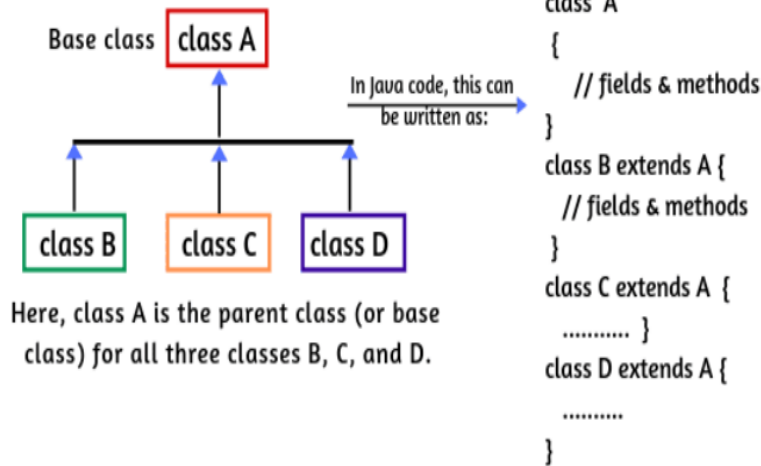


Fig: Hierarchical Inheritance in Java

Hierarchical Inheritance example

```

class ParentClass{
    int a;
    void setData(int a) {
        this.a = a;}
}
class ChildClass extends
    ParentClass{
    void showData() {
        System.out.println("Inside
        ChildClass!");
        System.out.println("Value
        of a is " + a);
    }
}
  
```

```

class ChildClassToo extends
    ParentClass{
    void display() {

        System.out.println("Inside
        ChildClassToo!");

        System.out.println("Value
        of a is " + a);
    }
}
  
```

Hierarchical Inheritance example

```
public class HierarchicalInheritance {
    public static void main(String[] args) {
        ChildClass child_obj = new ChildClass();
        child_obj.setData(100);

        child_obj.showData();
        ChildClassToo childToo_obj = new ChildClassToo();
        childToo_obj.setData(200);
        childToo_obj.display();
    }
}
```

```
Inside ChildClass!
Value of a is 100
Inside ChildClassToo!
Value of a is 200
```

Multiple Inheritance

- Deriving a subclass from more than one superclass is known as **multiple inheritance** in Java.
- In other words, when a class inherits features from multiple parent types, it is referred to as multiple inheritance.
- However, Java does not support multiple inheritance through classes. This means a class cannot extend more than one class in Java. Attempting something like this is illegal in Java:
- **class C extends A, B { } // Not allowed**
- Java supports only single inheritance through class where a class can extend only one class. Java supports multiple inheritance only through interfaces, where a class can implement multiple interfaces safely.

Multiple Inheritance

- Deriving a subclass from more than one superclass is known as **multiple inheritance** in Java.

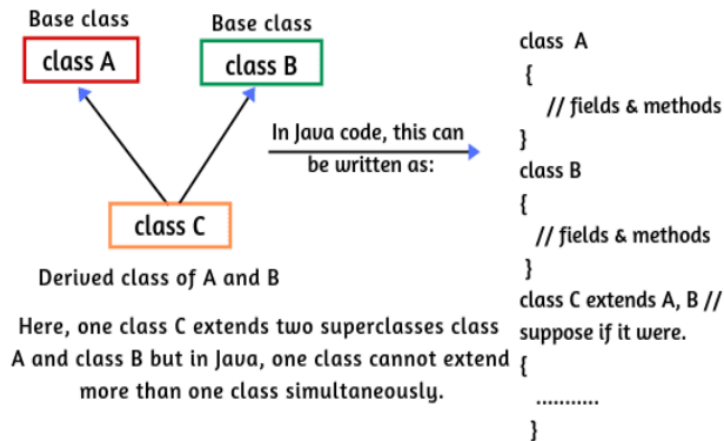


Fig: Multiple Inheritance in Java

multiple Inheritance example [Main.java]

```
interface Animal {
    void eat();
}
interface Bird {
    void fly();
}
class Bat implements Animal, Bird
{
    public void eat() {
        System.out.println("Bat eats insects.");
    }
    public void fly() {
        System.out.println("Bat can fly.");
    }
}
```

multiple Inheritance example [Main.java]

```
public class Main {
    public static void main(String[] args) {
        Bat bat = new Bat();
        bat.eat();
        bat.fly();
    }
}
```

```
Bat eats insects.
Bat can fly.
```

Hybrid Inheritance

- A **hybrid inheritance** in Java is a combination of single and multiple inheritance. It can be achieved in the same way as multiple inheritance using interfaces in Java.

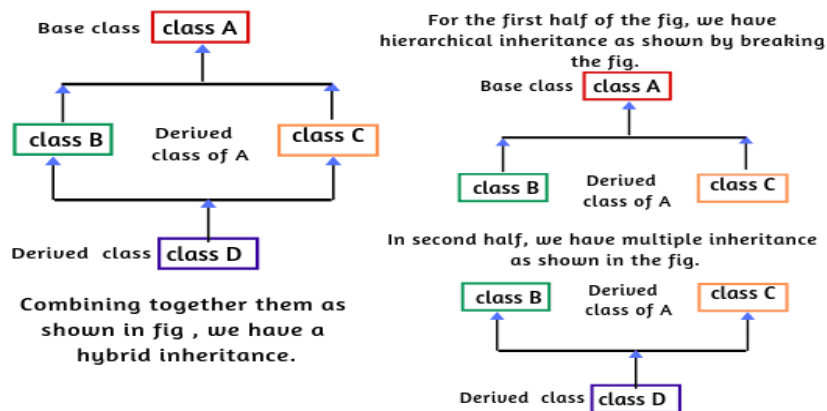


Fig: Hybrid Inheritance in Java

Hybrid Inheritance example

```
interface A {
    void methodA();
}
interface B extends A {
    void methodB();
}
interface C extends A {
    void methodC();
}
```

```
class D implements B, C {
    public void methodA() {
        System.out.println("Method A");
    }
    public void methodB() {
        System.out.println("Method B");
    }
    public void methodC() {
        System.out.println("Method C");
    }
}
```

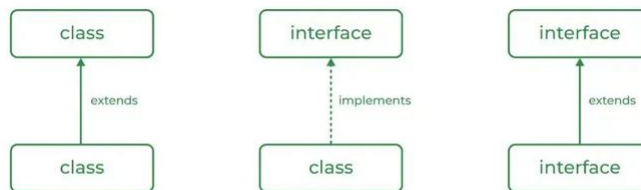
Hybrid Inheritance example

```
public class Main {
    public static void main(String[] args) {
        D obj = new D();
        obj.methodA();
        obj.methodB();
        obj.methodC();
    }
}
```

```
Method A
Method B
Method C
```

Interface and class in java

- In Java, a class is a blueprint for creating objects with both state (fields) and behavior (methods), while an interface is a contract that defines a set of behaviors a class must implement, primarily for achieving abstraction and multiple inheritance of type.
- The interface can have methods and variables, but the methods declared in the interface are by default abstract (only method signature, nobody).



Key Difference : Class vs. Interface

Features	Class	Interface
Keyword	The keyword used to create a class is " class ".	The keyword used to create an interface is " interface ".
Instantiation	A class can be instantiated, i.e., objects of a class can be created using new .	An interface cannot be instantiated directly, instead, it is implemented by a class.
Inheritance	Classes do not support multiple inheritance.	Interface supports multiple inheritance.
Inheritance Mechanism	A class can inherit another class using the keyword extends .	A class uses implements to adopt an interface, while an interface uses extends to inherit another interface.

Key Difference : Class vs. Interface

Features	Class	Interface
Constructors	It can contain constructors.	It cannot contain constructors.
Methods	Methods in a class can be abstract, concrete, or both.	An interface contains abstract methods by default (before Java 8) or default/static methods (from Java 8 onward).
Access Specifiers	Variables and methods in a class can be declared using any access specifier(public, private, default, protected).	All variables and methods in an interface are declared public

Key Difference : Class vs. Interface

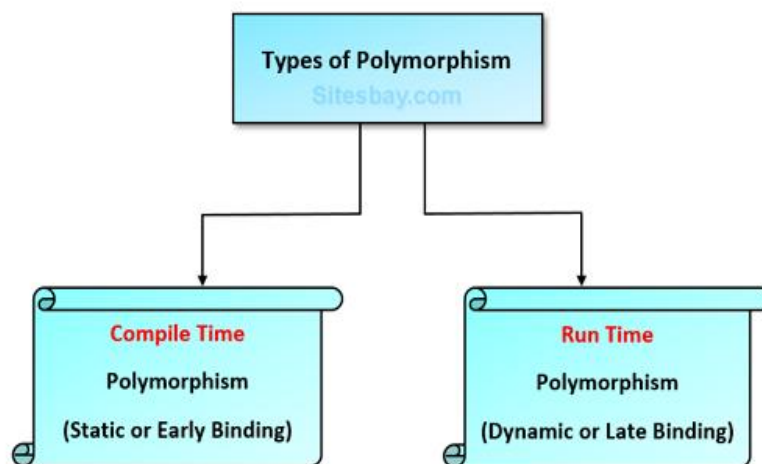
Features	Class	Interface
Variables	Variables in a class can be static, final, or neither.	All variables are static and final i.e. constant.
Purpose	A class is a blueprint for creating objects and encapsulates data and behaviour.	An interface specifies a contract for classes to implement by focusing on capabilities rather than implementation.
Relationship	Represents an "is-a" relationship. (e.g., a Car is a Vehicle)	Represents a "can-do" or "has-a" capability (eg. A dog can implement the Runnable interface).

Polymorphism

- In java programming the Polymorphism principal is implemented with method overriding concept of java.
- Polymorphism principal is divided into two sub principal they are:
 - Static or Compile time polymorphism
 - Dynamic or Runtime polymorphism
- **Static polymorphism** in Java is achieved by method overloading and **Dynamic polymorphism** in Java is achieved by method overriding
- **Note:** Java programming does not support static polymorphism because of its limitations and java always supports dynamic polymorphism.



Polymorphism



Overloading and overriding of methods

Overriding

```

class Dog{
    public void bark(){
        System.out.println("woof ");
    }
}

class Hound extends Dog{
    public void sniff(){
        System.out.println("sniff ");
    }

    public void bark(){
        System.out.println("bowl");
    }
}

```

Same Method Name,
Same parameter

Overloading

```

class Dog{
    public void bark(){
        System.out.println("woof ");
    }

    //overloading method
    public void bark(int num){
        for(int i=0; i<num; i++)
            System.out.println("woof ");
    }
}

```

Same Method Name,
Different Parameter

Overloading and overriding of methods

- **Overloading** occurs when two or more methods in one class have the same method name but different parameters.
- **Overriding** occurs when two methods have the same method name and parameters. One of the methods is in the parent class, and the other is in the child class.
- Overriding allows a child class to provide the specific implementation of a method that is *already* present in its parent class.
- Polymorphism applies to overriding, not to overloading.

Overloading vs overriding

Overloading

- Must have at least two methods by the same name in the class.
- Must have a different number of parameters.
- If the number of parameters is the same, then it must have different types of parameters.
- Overloading is known as **compile-time polymorphism**.

VS

Overriding

- Must have at least one method by the same name in both parent and child classes.
- Must have the same number of parameters.
- Must have the same parameter types.
- Overriding is known as **runtime polymorphism**.

Overloading example

```
public class Test {
    public static void main(String args[]) {
        MethodOverloadingExample ol = new
        MethodOverloadingExample();
        ol.show("Be in Present");
        ol.show(10, "Be in Present");
    }
}

class MethodOverloadingExample{
    public void show(String s) {
        System.out.println(s);
    }
    public void show(int i , String s) {
        System.out.println(i + " " + s);
    }
}
```

Output

```
Be in Present
10 Be in Present
```

Overriding example

```
public class MethodOverriding1 {
    public static void main(String args[])
    {
        // If a Parent type reference refers to a Parent object, then
        //Parent's isplay() is called
        Parent parentObject = new Parent();
        parentObject.display();

        // If a Parent type reference refers to a Child object Child's
        //display() is called. This is called RUN TIME POLYMORPHISM.
        Parent childObject = new Child();
        childObject.display();
    }
}
```

Overriding example

```
//Base class
class Parent {
    public void display() {
        System.out.println("parent method is executed");
    }
}

//Derived or Inherited class
class Child extends Parent {

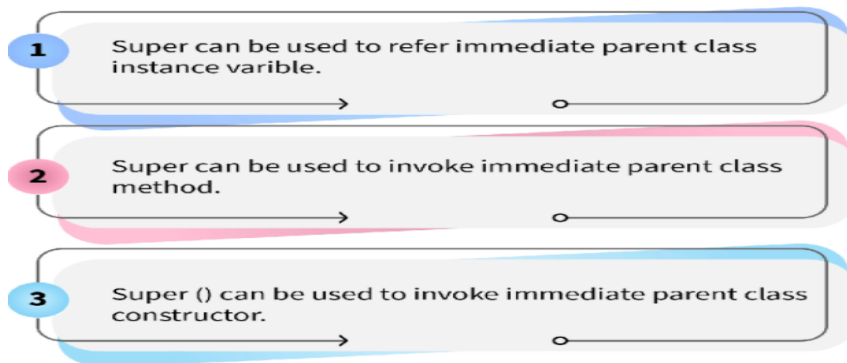
    //Below method overrides the Parent display() method
    //@Override
    public void display() {
        System.out.println("child method is executed");
    }
}
```

Output
parent method is executed
child method is executed

Super keyword

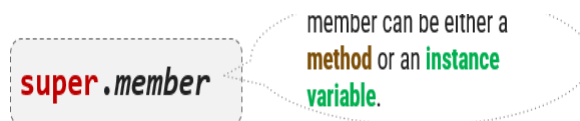
- The **super** keyword in Java is used as a reference variable to access objects from parent classes.
- It allows access to parent class constructors, members, and methods in the derived class.
- The keyword plays a crucial role in inheritance and polymorphism concepts.

Usage of Super Keyword



Super keyword

- **To Refer Immediate Parent Class Instance Variable:**
 - In this scenario, we have two classes: **School** and **Teacher**, which store information about schools in a specific region and the teachers working in those schools. The **Teacher** class is a subclass of the **School** class.
 - Both classes have a name variable of type String. We will now explore how to access the name variable of the parent class, School.



To Refer Immediate Parent Class Instance Variable Example

```
// Parent class
class School {
    // Instance variables
    int id;
    String name = "DDU Nadiad";
}
```

To Refer Immediate Parent Class Instance Variable Example

```
class Teacher extends School {
    // Instance variable in the child class
    int id;
    String name;
    void printSchoolName() {
        // Using super.name, we can access the name variable of the parent class
        // Using this.name, we can access the name variable of the child class
        System.out.println("School Name: " + super.name);
    }
    public static void main(String[] args) {
        Teacher ob = new Teacher();
        ob.printSchoolName();
    }
}
```

School Name: DDU Nadiad

Super keyword

- **To Invoke Immediate Parent Class Method:**
 - Now, both the base and derived classes contain the same method, **printID()**.
 - Let's see how we can access the **printID()** method of the parent class inside the child class.

To Invoke Immediate Parent Class Method example

```
class School {  
    // Instance variables  
    int id = 1;  
    String name = "DDU Nadiad";  
    void printID() {  
        System.out.println("School ID: " + this.id);  
    }  
}
```

To Invoke Immediate Parent Class Method example

```
class Teacher extends School {
    // Instance variable in the child class
    int id = 10;
    void printSchoolName() {
        System.out.println("School Name: " + super.name);
    }
    void printID() {
        System.out.println("Teacher ID: " + this.id);
        super.printID();
    }
    public static void main(String[] args) {
        Teacher ob = new Teacher();
        ob.printID();
    }
}
```

```
Teacher ID: 10
School ID: 1
```

Super keyword

- **To Invoke Immediate Parent Class Constructor:**
 - The **super** keyword can invoke both the default and parameterized constructors of the parent class in the child class.
 - This is useful when we want to initialize the instance members of the parent class from the child class.

To Invoke Immediate Parent Class Constructor example

```
// Base class Person
class Person {

    Person() {
        System.out.println("Person class Constructor");
    }
}
```

To Invoke Immediate Parent Class Constructor example

```
// Derived class Teacher
class Teacher extends Person {
    Teacher() {
        // Invoke the parent class constructor
        super();
        System.out.println("Teacher class Constructor");
    }
}

class Main {

    public static void main(String[] args) {
        // Invoke the constructor of the Teacher class
        Teacher s = new Teacher();
    }
}
```

Person class Constructor
Teacher class Constructor

To Invoke Immediate Parent Class Constructor example

- In Java, we can call the parameterized constructor of the parent class from the subclass using the Super Keyword.

```
class Birds {
    int age;
    Birds(int age) {
        this.age = age;
        System.out.println("Birds class constructor invoked!");
    }
}
```

To Invoke Immediate Parent Class Constructor example

```
public class Parrot extends Birds {
    String name;
    Parrot(String name, int age) {
        super(age);
        this.name = name;
        System.out.println("Parrot class constructor invoked!");
    }
    void printDetails() {
        System.out.println("Name: " + this.name);
        System.out.println("Age: " + this.age);
    }
    public static void main(String[] args) {
        Parrot ob = new Parrot("Teepee", 5);
        ob.printDetails();
    }
}
```

```
Birds class constructor invoked!
Parrot class constructor invoked!
Name: Teepee
Age: 5
```

